

YoHoBed 2.0

Property Management System — Client Demo Walkthrough & Feature Guide

Prepared 23 July 2026 · MVP build (compartments A–K complete) · 105 automated tests green · All data below is live in the local demo environment

THE ONE-PARAGRAPH PITCH

YoHoBed 2.0 is a complete rebuild of the property-owner Extranet as a modern, multi-tenant Property Management System: one place where a Sri Lankan hotel owner runs their entire operation — rates & availability, bookings and the front desk, OTA channel distribution, finance & settlement, guest communications, reviews and promotions — with YoHo staff operating a separate cross-tenant console. It preserves the legacy platform's pricing logic to the cent while fixing its four known production bugs, and every screen below is running on the new stack right now.

DEMO ENVIRONMENT (ALREADY RUNNING)

Owner app (main demo)	<code>http://localhost:3000</code> — sign in as <code>owner@demo.yohobed.test</code> / <code>password123</code>
Staff console	Same URL, sign in as <code>staff@yohobed.test</code> / <code>password123</code> (use a second browser window / incognito)
API	<code>http://localhost:3001</code> (NestJS) — the web app talks to this; you never open it directly
Demo tenant	Cinnamon Lakeside Group — 2 properties: <i>Cinnamon Lakeside</i> (Deluxe Room ×5, simple 10% commission, no taxes) and <i>Ceylon Tax Villa</i> (Ocean Suite ×3, slab commission + 10% service charge + 15% VAT)
Data window	Rooms open & priced 15 Jul → 31 Aug 2026 · Rs 18,000 base weekdays / Rs 25,000 weekends

WHAT'S LOADED FOR YOU TO SHOW

Area	Ready-made demo data
Front desk (today)	1 arrival to check in live (Kasun), 1 departure to check out live (Sofia), 2 guests in-house, 2 bookings awaiting approval
Bookings	13 bookings covering <i>every</i> status: Pending, Approved, Checked-in, Checked-out, Cancelled, No-show — plus a coupon booking and a referral booking
OTA Inbox	2 imported OTA reservations (booking.com + agoda, auto-approved) and 1 <i>failed</i> import (expedia) for the live "Retry" moment
Finance	1 invoice paid in full, 1 invoice 50% advance-paid, July payout statement that reconciles to the cent, saved payout snapshot, settlement bank account on file

Deals	"August Getaway – 15%" promotion painted on the calendar, coupons SUMMER10 & FAMILY5000, referral partner Island Travels (5%)
Reviews / Comms	Two real guest reviews (5★ and 4★) submitted through the public review link; ~29 unread notifications; a full outbound email log
Staff console	A brand-new signup "Villa Serendib" waiting for staff approval, plus a pending booking to approve on the owner's behalf

SUGGESTED 20-MINUTE STORYLINE

1	Sign in, product shell, dark/light theme	1 min
2	Dashboard — run the front desk: check-in & check-out live	3 min
3	Calendar — prices, availability, bulk edits, promotions overlay, change history	3 min
4	Bookings — approve a pending booking, open the per-night price snapshot, amend	3 min
5	OTA Inbox — imported reservations + the failed-import Retry story	2 min
6	Finance — revenue, reconciling payout statement, invoices & payments	3 min
7	Deals · Comms · Reviews · Customers — quick lap	3 min
8	Staff console — approve the new hotel signup, cross-tenant powers, audit trail	2 min

Sections 9–12 (Setup, Profile, staff console, engineering story) are your back-pocket material for Q&A. Sections 13–17 are the **delivery plan**: what's left to build, what we need from your side, timeline, hosting & infrastructure, scalability and the budget reality.

1. Sign in & the product shell

What it is: secure email/password sign-in with self-serve registration, forgotten-password recovery, and a role-aware landing: owners land on their PMS, YoHo staff land on the staff console.

GUIDED STEPS

1. Open `http://localhost:3000`. You land on the sign-in screen (note the clean dark theme — this is the product default).
2. Point out the links: *Register* (self-serve hotel signup — you'll return to this in the staff section) and *Forgot password* (emails a reset link valid for 60 minutes).
3. Sign in as `owner@demo.yohobed.test` / `password123`.
4. You arrive on the **Dashboard**. Walk the left sidebar top-to-bottom — this is the whole product: *Dashboard, Calendar, Bookings, Inbox, Customers, Deals, Finance, Reviews, Comms, Setup, Profile*.
5. Click the **sun/moon toggle** in the sidebar footer to flip to light mode and back — every screen supports both.

THE LOGIC UNDERNEATH

Passwords are bcrypt-hashed; a successful login issues a signed JWT that carries the user's memberships (which hotels they belong to, and their role). Every subsequent request is checked twice: the JWT proves *who* you are, and a tenant guard proves the hotel you're acting on is *yours*. Login errors are deliberately identical for "unknown email" and "wrong password" so attackers can't discover which emails exist. A suspended hotel is blocked at sign-in *and* on every request — suspension takes effect immediately, not at next login.

WOW MOMENT

"One login, two products": after the owner tour, sign in as `staff@yohobed.test` in a second window — same login screen, completely different application (the staff console). The system routes by role.

2. Dashboard — the front desk

What it is: the receptionist's home screen. One glance answers: who arrives today, who leaves, who's in the building, what needs my approval, how full are we, and how much money is this month bringing in.

GUIDED STEPS

1. Read the stat tiles across the top: **Arrivals 1 · Departures 1 · In-house 2 · Occupancy % · Month gross**. All computed live for today (23 Jul).
2. The **Pending** banner shows **2 bookings awaiting approval** — you'll approve one in the Bookings section.
3. In **Arrivals today**: *Kasun Jayawardena* (ref 2607230005, 3 nights, Deluxe Room). Tap **Check in** — one tap, status flips to **Checked-in**, and In-house becomes 3.
4. In **Departures today**: *Sofia Marchetti* (ref 2607230004). Tap **Check out** — status flips to **Checked-out**... and the system has just emailed Sofia a review invitation automatically (you'll show the proof in Comms, §8).
5. Scroll the **Recent bookings** list — the newest activity, each with its status pill.
6. Use the property filter to flip between *Cinnamon Lakeside* and *Ceylon Tax Villa* — every number recomputes per property.

THE LOGIC UNDERNEATH

- **Occupancy %** = confirmed room-nights ÷ physical rooms for the day. Only "real" statuses count (Approved / Checked-in / Checked-out) — pending and cancelled bookings never inflate the number.
- **Month gross** is summed from each booking's *per-night price snapshot*, so a stay that straddles two months lands in the right month, night by night.
- **Check-in / check-out are guarded state transitions** (only an Approved booking can check in; only a Checked-in one can check out) with timestamps stamped for the audit trail. Check-out is also the trigger that mints the guest-review invitation — no staff action needed.

WOW MOMENT

The whole morning routine of a receptionist — arrivals, departures, approvals — is a series of single taps from one screen. In the legacy system there was no dashboard at all.

3. Calendar — rates, availability & restrictions

What it is: the revenue-manager's cockpit. A full-month wall calendar where every cell shows the *selling price* the guest will actually pay, the rooms left to sell, and open/closed status — with inline editing and bulk tools.

GUIDED STEPS

1. Open **Calendar**. Select *Cinnamon Lakeside* → *Deluxe Room*. July is on screen; every priced day shows ~**Rs 24,390.25** weekdays and ~**Rs 33,875.35** weekends.
2. Click any future cell and **edit inline** — change the base price and watch the selling price recompute instantly from the pricing engine (not a manual field!).
3. Step to **August**. Point out **15–20 Aug**: cells carry a *–15% drop overlay* — that's the "August Getaway" promotion you'll show in Deals (§7), already painted onto the calendar.
4. Point at **10 Aug**: only **1 room left** — your segue to the overbooking story ("the old system could sell this room twice; this one cannot" — §14).
5. Use the **bulk toolbar**: pick a From/To range (or "Whole month") and set price, open/close days, apply a last-minute drop, or set **min/max stay** in one action. (14–16 Aug already carries a min-stay of 2 nights — try to book a 1-night stay there later and the system politely refuses.)
6. Expand **Change history** at the bottom: every price, availability, drop and restriction change — what changed, for which dates, and *who did it*.
7. Switch to *Ceylon Tax Villa* → *Ocean Suite*: weekday cells read **Rs 31,625** — same Rs 18,000 base, but this property has slab commission and taxes baked into its selling price. 1–3 Aug additionally shows a 15% last-minute drop.

THE LOGIC UNDERNEATH — THE PRICING ENGINE

The owner only ever types the **base price** (what the hotel wants to keep). The system derives what the guest pays — in this exact order, matching the legacy platform to the cent:

1	YoHo commission	percentage: $\text{base} \div (1 - \text{pct}/100) - \text{base}$	e.g. $18,000 \div 0.9 - 18,000 = 2,000$
		or slab: fixed fee by base-price band	e.g. Rs 0–20,000 → 2,500
2	OTA gross-up Deluxe selling	$(\text{base} + \text{commission}) \div (1 - 18/100)$	$20,000 \div 0.82 = 24,390.25$ ←
3	Taxes (if any)	$\times (1+\text{SC}\%) \times (1+\text{NBT}\%) \times (1+\text{VAT}\%)$ in priority order $25,000 \times 1.10 \times 1.15 = 31,625$	← Ocean Suite selling
4	Last-minute drop	charged = $\text{selling} \times (1 - \text{drop}\%)$	$31,625 \times 0.85 = 26,881.25$

- All rounding is **round-up to 2 decimals** (ceiling) — the legacy behaviour, reproduced deliberately so migrated hotels see identical prices.
- Ceylon Tax Villa's weekday cell proves the whole chain: 18,000 base + 2,500 yoho + 4,500 ota + 6,625 taxes = 31,625. Nothing is invented; the split reconciles by construction — the same split Finance uses for settlement (§6).
- Every calendar edit also (a) writes an **ARI history** row with the actor's email, and (b) queues a **channel-manager sync event** in the same database transaction — so OTAs are told about every change, guaranteed (§14, bug #3).

4. Bookings — the reservation book

What it is: every reservation in one table — walk-ins created here and OTA bookings imported automatically — with the full lifecycle: approve, reject, cancel, no-show, check-in, check-out, and amend.

GUIDED STEPS

1. Open **Bookings**. Sweep the status pills — the book already contains every state: Pending Dinesh Fernando & Priya Sharma · Approved Chen Wei, Aisha Rahman, Nadeesha Silva... · Checked-in Emma Whitfield · Checked-out Ruwan Perera · Cancelled Liam O'Connor · No-show Harsha Wickramasinghe.
2. **Approve live:** find *Dinesh Fernando* (ref 2607230009, 3–6 Aug) and tap **Approve** — Pending → Approved. (Leave *Priya Sharma* pending — the staff console will approve that one, \$10.)
3. **Open a booking** — use *Aisha Rahman* (ref 2607230011): a proper detail card with the guest, the stay, the **per-night price breakdown** (7 Aug Rs 24,390.25 · 8–9 Aug Rs 33,875.35 each = Rs 92,140.95) and her **SUMMER10 coupon discount of Rs 9,214.09** recorded on the booking — plus the lifecycle **trail** (created → approved, timestamped).
4. **Create a walk-in:** tap + *Book a stay*. A full modal: guest name/email/phone, dates, number of rooms, live nights×rooms summary, and a coupon field. Book any August stay — it prices itself from the calendar and lands as Pending, and the guest gets a confirmation email instantly (\$8 shows the email log).
5. **Amend:** open any Pending/Approved booking → *Edit*. Change the dates — the system re-prices the new nights and swaps the inventory atomically; if the new dates don't fit, it refuses with a clear conflict message rather than overbooking.
6. Show **Cancel** on a pending booking: the room instantly returns to the calendar's sellable count.

THE LOGIC UNDERNEATH

- **One transaction does everything.** Creating a booking: verifies the room/occupancy pairing, checks min/max-stay rules for the arrival date, snapshots tonight's prices night-by-night, atomically reserves inventory (impossible to oversell — the database refuses if any night lacks rooms), applies coupon/referral, generates the reference from an atomic daily counter (format `yymmdd####` — today's are 260723xxxx), queues the OTA sync event, notifies the owner, and emails the guest. If any step fails, *everything* rolls back.
- **Prices are snapshotted, not referenced.** The per-night breakdown you showed is frozen at booking time — repricing the calendar tomorrow never rewrites yesterday's contracts. Finance settles from these snapshots.
- **The status machine is strict.** Cancel/reject release inventory; no-show keeps it consumed (the room was held all night — that's revenue, matching legacy policy); only Approved can check in. Every transition is recorded in the trail.

5. Inbox — OTA reservations

What it is: where bookings made on booking.com, Agoda, Expedia etc. arrive via the channel manager. The core promise: **an OTA reservation can never be silently lost** — every push is recorded first, then imported; failures stay visible with a Retry button.

GUIDED STEPS

1. Open **Inbox**. Two clean imports: *Hannah Lee* (booking.com, 12–14 Aug, Rs 48,780.50, ref 2607230014) and *Marco Rossi* (agoda, 20–22 Aug, Rs 63,250, ref 2607230015). Both were imported as **Approved** bookings automatically — flip to Bookings and they're in the book, inventory already decremented.
2. Point at the **failed** row: *Yuki Tanaka* (expedia, 2–4 **Sep**). The error is displayed: those September nights have no prices yet. The reservation was *not lost* — it's sitting here waiting.
3. **Live fix (the money moment):** go to Calendar → Deluxe Room → September → set a price for 1–7 Sep and open the days. Return to Inbox → tap **Retry** → the reservation imports and appears in Bookings. A failed OTA booking, recovered in 20 seconds by the owner, no support ticket.
4. Show the **room-code mappings** panel: CM-DLX-001 → Deluxe Room, CM-OCN-101 → Ocean Suite — how the channel manager's room codes route to real rooms.

THE LOGIC UNDERNEATH

- The channel manager calls a secured webhook (shared-secret header; wrong secret → rejected). The room code alone identifies the hotel — and the payload is **recorded before import is attempted**, so a crash mid-import still leaves the reservation on file.
- Imports are **idempotent** per channel + external reference — if booking.com retries the same push five times, you get one booking, not five.
- OTA bookings import as **auto-approved** (the guest already paid the OTA — there is nothing for the owner to approve) and consume inventory through the exact same atomic path as walk-ins. An OTA cancellation releases the rooms automatically.
- An unmapped room code answers with a specific "retry later" signal to the channel manager instead of swallowing the booking.

6. Finance — revenue, invoices & settlement

What it is: the money screen. Monthly revenue by status, guest invoices and payments, and the **payout statement** — the settlement document that splits every rupee of gross revenue between the hotel, YoHo, the OTA channel, and the taxman, and proves the split adds back up.

GUIDED STEPS

- 1. Open **Finance**. The month picker defaults to **July 2026**. Revenue summary: confirmed gross ~**Rs 606,242** across the tenant, with a by-status breakdown (point out that Cancelled contributes zero).
- 2. Select property *Cinnamon Lakeside* and show the **Payout statement** for July:

Gross selling	= Hotel base (net payable)	+ YoHo commission	+ OTA commission	+ Taxes
Rs 345,528.55	Rs 255,000.00	Rs 28,333.34	Rs 62,195.21	Rs 0.00

...and the "✓ **reconciles**" line: the four parts sum to the gross *to the cent*. The hotel's payout is the base — exactly what the owner typed on the calendar. Flip to *Ceylon Tax Villa* to show a statement where the taxes column is non-zero.

- 3. **Invoices:** open INV-2607230001 (Ruwan Perera, 3 nights) — line items from the per-night snapshot, status **paid**, with its card payment (VISA-4212) recorded in full. Then INV-2607230007 (Tharindu Fernando) — a **50% advance** received by bank transfer, invoice still open. Record the balance live if you like: the invoice flips to paid automatically the moment received payments cover it.
- 4. Show the saved **July payout snapshot** — statements can be frozen as payout records for the payment run.
- 5. Point to **Profile** → **bank details** (\$9): Commercial Bank of Ceylon, Kollupitiya — the settlement knows *where* to send the money.

THE LOGIC UNDERNEATH

- The settlement split is not an approximation — it reuses the *same decomposition* the pricing engine used when each night was priced (\$3): base + yoho + ota + taxes = gross, per night, snapshot on the booking. Because settlement reads those snapshots, the statement reconciles **by construction** — there is no "rounding difference" bucket.
- Revenue counts **Approved, Checked-in and Checked-out** bookings (checked-in/out are earned revenue); Pending, Cancelled and Rejected never touch the money.
- Invoices are numbered INV-<booking reference> and are idempotent — asking twice returns the same invoice, never a duplicate.

WOW MOMENT

"Every rupee is accounted for": gross → hotel + YoHo + OTA + tax, reconciling to the cent, live, per property and period. This is the trust argument against the legacy system's spreadsheet reconciliation.

7. Deals — promotions, coupons & referral partners

What it is: the commercial toolkit: date-window promotions that discount the calendar, coupon codes guests redeem at booking, and referral partners who earn a commission on the bookings they bring.

GUIDED STEPS

1. Open **Deals**. Show the promotion "**August Getaway**" – **15% (15–20 Aug, Cinnamon Lakeside)** — then remind the client you already *saw* it as the drop overlay on the August calendar (§3). Creating a promotion and tapping *Apply* paints the discount across every rate of the property for the window, and notifies the OTA channels.
2. Show the coupons: **SUMMER10** (10%, valid 1 Jul–31 Aug, unlimited) and **FAMILY5000** (Rs 5,000 fixed, capped at 20 uses, Cinnamon Lakeside only). Tie it back to Aisha Rahman's booking (§4): SUMMER10 recorded a **Rs 9,214.09** discount on her Rs 92,140.95 stay.
3. Show referral partner **Island Travels (Pvt) Ltd**, code `ISLAND`, 5% — and the commission ledger: *Chen Wei's* 3-night Ocean Suite booking (ref 2607230008) earned Island Travels a pending commission automatically.

THE LOGIC UNDERNEATH

- **Promotions** are implemented as last-minute drops on the rate calendar — one mechanism, no special cases: the charged price is always $\text{selling} \times (1 - \text{drop}\%)$, and removing a promotion later doesn't retroactively rewrite prices already painted (you re-price to undo).
- **Coupons** validate at booking time: active, inside its date window, under its usage cap, and compatible with the property. The discount is stored on the booking itself, so amendments keep the original deal.
- **Referral commissions** compute on the commissionable amount (not on taxes) and accrue per booking as pending → paid — an auditable ledger per partner.

8. Comms — notifications, emails & templates

What it is: the communication centre: the in-app notification bell, the full outbound email log (every mail the system sent, with its rendered content), and editable per-language message templates.

GUIDED STEPS

1. Point at the **bell** — ~29 unread notifications built up during the data load: *booking created, OTA booking imported, review received...* Open the panel, mark one read, then *mark all read* — the badge clears.
2. Open **Comms** → **Messages**: the outbound email log. Show Sofia Marchetti's **review invitation** — sent automatically the moment you checked her out on the Dashboard (\$2), containing her one-time review link. Show a booking confirmation (e.g. Hannah Lee's OTA import) with the amount rendered.
3. Open **Templates**: booking confirmation, review invite, etc., in **English and Sinhala**, with `{{placeholder}}` variables. Edit a word, save — the next email uses it immediately.

THE LOGIC UNDERNEATH

- Emails are **queued inside the same transaction** as the event that caused them, then delivered after commit — a failed booking never sends a confirmation, and a sent confirmation always has a booking behind it. Failures are recorded on the message with the error, visible in this log.
- Templates are **per hotel and per language** (en/si/ta seeded). Every new hotel gets the full starter set automatically at registration.
- The email provider sits behind a seam: the demo logs to the console; production flips one environment variable to send through Resend. Nothing else changes.

9. Reviews · Customers · Setup · Profile

REVIEWS — VERIFIED GUEST FEEDBACK

1. Open **Reviews**: *Ruwan Perera* ★★★★★ "Superb lakeside stay..." and *Ayesha Khan* ★★★★★ "Beautiful ocean views...", with per-property averages.
2. Full-circle demo: from Comms, copy Sofia's review link (`/review/<token>`), open it in a private tab — the public star form knows her stay. Submit — the review appears here instantly and the bell rings.

LOGIC

Invites are minted only by a real check-out, are single-use (a second submit is refused), and need no guest account — the unguessable token in the link is the authorisation. Reviews are therefore verified-stay by construction; nobody can review a hotel they didn't sleep in.

CUSTOMERS — THE GUEST DIRECTORY

1. Open **Customers**: every guest with bookings count, nights, confirmed spend, last check-in. Expand *Ruwan Perera* for his full history. Repeat guests are recognised by email and accumulate — the base for loyalty marketing.

SETUP — PROPERTIES, ROOMS & RATES

1. Open **Setup**: the two properties with their rooms (Deluxe Room ×5, Ocean Suite ×3), room types, rate plans (meal plans RO/BB/HB/FB/AI) and occupancies, property & room **photos** (upload live if you like), and the CM room-code mappings.

LOGIC

The hierarchy is `property → room → rate plan (meal plan) → occupancy` ; the calendar prices at occupancy level. Commission model (10% vs slabs) and the property's tax profile are configured per property — that's the entire difference between the two demo properties' prices.

PROFILE — ACCOUNT & SETTLEMENT DETAILS

1. Open **Profile**: the settlement bank account (*Commercial Bank of Ceylon, Kollupitiya, a/c 8001234567, SWIFT CCEYLKLX*) — where payouts (\$6) get paid; platform-agreement acceptance; change-password.

10. Staff console — YoHo operations

What it is: the separate cross-tenant application for YoHo's own team: onboard new hotels, act on any tenant's bookings, suspend/reactivate accounts — with every action written to an immutable audit trail.

GUIDED STEPS

1. In a **second browser window (incognito)**, sign in as `staff@yohobed.test` / `password123` → you land on the **Staff console**, not the owner PMS.
2. The tenant list shows **Cinnamon Lakeside Group** active (with its pending-booking count) and **Villa Serendib** pending — a hotel that self-registered and is waiting for activation.
3. **Approve Villa Serendib** — one click: pending → active, and the owner (Shanika De Silva) automatically receives the welcome email (visible in the API console log).
4. Open Cinnamon Lakeside's bookings and **approve Priya Sharma's** pending booking (ref `2607230010`) *on the owner's behalf* — cross-tenant support in action. Flip to the owner window: the booking is Approved.
5. Demonstrate **Suspend** on a tenant (and reactivate) — suspension blocks sign-in *and* kills in-flight sessions on their next request.
6. Open the **Audit trail**: every staff action you just took — actor, action, entity, timestamp.

THE LOGIC UNDERNEATH

- Staff powers come from a **role** (`YOHO_STAFF` / `YOHO_ADMIN`) on the user's membership — an owner calling a staff route gets a hard 403. There is no "admin email list" backdoor like the legacy system had.
- Even staff reads run under the same row-level security as owners — the console explicitly scopes into one tenant at a time; there is no "god query" path that could leak one hotel's data into another's screen.
- Onboarding is **self-serve + staff approval**: hotels register themselves (they can sign in and set up while pending), staff activate — the activation is the trust gate.

11. Under the hood — the engineering story (Q&A ammunition)

FOUR LEGACY PRODUCTION BUGS — FOUND, FIXED, AND PROVEN WITH TESTS

Legacy bug	What went wrong	The 2.0 fix
#1 Overbooking race	Two simultaneous bookings for the last room could both succeed (read-then-write with no lock).	Atomic conditional decrement in the database — proven with a test firing 12 parallel bookings at 1 room: exactly one wins, count never goes negative.
#2 Wrong price key	Walk-ins priced by the wrong identifier (room id used as the rate key) — wrong amounts billed.	Bookings carry the explicit occupancy (pricing) key, verified to belong to the room.
#3 Silent channel-manager push	Price/availability pushes to OTAs were fire-and-forget: a failure was <i>silent</i> → stale OTA prices, overbookings.	Transactional outbox: the sync event commits <i>with</i> the change, a worker delivers with 5 retries + exponential backoff; final failures alert and stay visible (Inbox/health screens).
#4 Duplicate booking references	References generated by "max + 1" — concurrent bookings could collide.	Atomic per-day counter with a uniqueness guarantee (20-parallel-bookings test: all unique).

ARCHITECTURE IN FIVE BULLETS

- **Stack:** TypeScript end-to-end — NestJS API + Next.js web + background worker (BullMQ/Redis) + PostgreSQL 16, in a Turborepo monorepo. The pricing engine is a framework-free package shared by API, seed and tests.
- **Multi-tenant security is enforced by the database**, not by remembering to add WHERE clauses: PostgreSQL Row-Level Security with a default-deny policy. The app connects as a restricted role; a query without a tenant context returns *nothing*. Cross-tenant reads 404 by construction.
- **Parity by contract:** the legacy pricing formulas (round-up rounding, commission gross-up, ÷0.82 OTA factor, nested tax decomposition) are reproduced exactly and pinned by unit tests with the legacy's own numbers.
- **105 automated tests:** 42 domain (pricing parity), 8 database-integration (RLS isolation, concurrency races), 36 API end-to-end over real HTTP + real Postgres, 19 channel-manager contract tests — plus CI on every push.
- **Channel manager ready:** the AxisRooms adapter is code-complete against the legacy integration contract (same 5 endpoints, same wire format), running behind the outbox. It needs only production credentials to go live; the demo uses a stub so every sync is visible locally.

HONEST ANSWERS FOR "WHAT'S NOT IN YET"

- **Data migration (ETL)** from the legacy MySQL — designed (with a parity harness as the cutover gate), not yet run; today's system runs on fresh data.
- **Hosting/deploy** — runs locally/CI today; production infrastructure is the next step after sign-off.
- **AxisRooms live connection** — adapter finished, waiting on credentials and a test property.
- **Planned next (post-MVP):** the AI Supplier Copilot — a chat assistant with read-only, tenant-scoped tools over these exact services ("How was my August? Anything I should reprice?"). The service layer was built MCP-ready for it from day one.

12. Cheat sheet & recovery

LOGINS & URLS

Owner PMS	<code>http://localhost:3000 · owner@demo.yohobed.test / password123</code>
Staff console	<code>same URL · staff@yohobed.test / password123</code> (second window)
Pending signup	<code>Villa Serendib · shanika@villaserendib.lk / password123</code>

NUMBERS YOU CAN QUOTE WITH CONFIDENCE

Claim	Where to show it
Deluxe weekday Rs 24,390.25 = 18,000 base + 2,000 YoHo + 4,390.25 OTA	Calendar cell / any Deluxe booking detail
Ocean weekday Rs 31,625 = 18,000 + 2,500 slab + 4,500 OTA + 6,625 tax	Ceylon Tax Villa calendar / booking detail
July payout: Rs 345,528.55 gross → Rs 255,000 net payable, reconciles to the cent	Finance → Cinnamon Lakeside → July statement
Coupon SUMMER10 took Rs 9,214.09 off Aisha Rahman's Rs 92,140.95 stay	Bookings → 2607230011 detail
Every status in the book; today: 1 arrival, 1 departure, 2 in-house, 2 pending	Dashboard / Bookings
OTA reservation recovered live (failed → priced → Retry → imported)	Inbox (expedia / Yuki Tanaka row)

IF SOMETHING IS NOT RUNNING (FULL COLD START, ~2 MIN)

```
# 1. Start Docker Desktop, then in d:\YoHo Application\yohobed2 :
docker compose up -d

# 2. API (new terminal, from yohobed2\apps\api)
$env:APP_DATABASE_URL='postgresql://yoho_app:yoho_app_pw@localhost:5433/yohobed'
$env:JWT_SECRET='dev-jwt-secret-yohobed-2026'; $env:PORT='3001'
node dist/main.js

# 3. Web (new terminal, from yohobed2\apps\web-extranet)
node node_modules/next/dist/bin/next start -p 3000

# 4. Worker – optional, drains OTA sync events (new terminal, from yohobed2)
$env:APP_DATABASE_URL='postgresql://yoho_app:yoho_app_pw@localhost:5433/yohobed'
$env:REDIS_URL='redis://localhost:6380'
pnpm --filter @yohobed/worker start
```

RESET THE DEMO DATA TO EXACTLY THIS GUIDE

```
# from d:\YoHo Application\yohobed2 (wipes + rebuilds the demo tenant, ~30 s)
$env:DATABASE_URL='postgresql://postgres:postgres@localhost:5433/yohobed'
pnpm --filter @yohobed/db db:seed
node "<scratchpad>\demo-data.mjs" # the demo-data builder script
```

Re-seeding regenerates ids and booking references (they are date-based: `yymmdd####`) — the storyline stays identical, only reference numbers shift to the new date. A copy of the builder script is kept at `yohobed2\scripts\demo-data.mjs`.

13. Delivery plan — what's left to build

Everything you demonstrated in sections 1–10 is **built, tested (105 automated tests) and running**. The remaining work falls into three buckets: taking what exists *live* (hosting, channel manager, data migration), *completing* the staff back-office and polish items, and the *growth layer* (booking engine, analytics, AI suite). Component by component:

Component	Where it stands today	What remains
PMS / Extranet the owner product	Complete MVP — dashboard, calendar/ARI, bookings & front desk, OTA inbox, finance & settlement, deals, comms, reviews, customers, photos, onboarding. This is the demo.	Polish pack: Seasons management UI (API exists), a "My channels" screen, invoice PDF download, Sinhala/Tamil interface translation (framework is ready — templates already ship in 3 languages), Excel bulk-upload (keep-or-replace decision pending), multi-currency display (USD).
Backend portal staff console	Core built: tenant onboarding/approval, cross-tenant booking management, suspend/activate, full audit trail.	Full back-office: reference-data admin (cities, currencies, OTA master list), per-OTA commission configuration, revenue reconciliation screens & finance exports (Excel/PRS), campaign & bulk-notification engine (email/SMS/push), and — optional, later — the DMC/corporate B2B module.
Channel manager AxisRooms	Adapter code-complete , ported 1:1 from the legacy integration contract (same 5 endpoints, same wire format), 19 contract tests, retry/dead-letter/alerting proven.	Go-live only: production credentials, a joint certification test with AxisRooms, one pilot property synced in parallel with the legacy system, then switch-over. RateGain adapter is stubbed for later.
OTA distribution	Outbound sync (transactional outbox) + inbound reservation inbox both live and idempotent.	Per-OTA commission rates in settlement, a channel-health dashboard for owners, and (new capability) a rate-parity monitor.
Data migration (ETL)	Designed: legacy MySQL → new Postgres with a <i>parity harness</i> as the cutover gate (old and new systems must produce identical numbers before any hotel switches).	The largest remaining engineering item: field mapping, migration scripts, dry runs on a legacy copy, per-hotel parallel run, staged cutover in waves. Needs legacy DB access (§14).
Production platform	Runs locally + full CI pipeline on every commit.	Hosting, deploy pipeline to staging/production, domains & TLS, verified email sending domain, photo storage on cloud object storage, monitoring/alerting, tested backups. Plan in §16.
Growth layer post-launch	Not started (deliberately — launch first).	Direct booking engine + payment gateway (deposits/online payment), analytics module (occupancy, ADR, RevPAR trends) with a small data warehouse, guest messaging, mobile PWA.
AI suite the differentiator	The service layer was built AI-ready from day one (clean, tenant-scoped domain services).	Phased: Supplier Copilot (chat: "How was my August? What should I reprice?") → AI Revenue Manager (needs the analytics module first) → Distribution Healer (auto-diagnoses channel issues) → agentic distribution. Cost model in §17.

HOW TO SAY IT TO THE CLIENT

"The product you just saw is finished software. What's left is *plumbing to production* — servers, the AxisRooms hookup, moving your data — plus the staff back-office and the growth features we sequence together after launch."

14. What we need from the client's side

Nothing below is engineering — these are access, accounts and decisions. Each item gates a specific phase; the earlier we have them, the earlier that phase lands.

Item	Specifically	Unblocks
1 · AxisRooms credentials blocker	Production API credentials + confirmation of channel ID (164) + an AxisRooms contact for a certification test, and one pilot property we may sync.	Channel manager go-live (§13). Without this the CM runs code-complete but dark.
2 · Legacy database access blocker	Read-only access to the production MySQL (a nightly dump or replica is enough) + one person who can answer data questions and sign off the field mapping.	ETL build, parity testing, and every cutover date.
3 · Domain & DNS control	Access to the yohobed.com DNS (or the registrar) to create app/api subdomains and the email-authentication records (SPF/DKIM/DMARC).	Hosting with real URLs + genuinely deliverable email.
4 · Cloud & service accounts	A cloud account in the company's name (we set everything up inside it — you own the infrastructure and the bills), plus a Resend account for email. Budget approval per §17.	Production hosting, backups, email provider switch (one env var — code is ready).
5 · Pilot hotels & people	2–3 friendly properties for the parallel-run pilot; your staff for a training session (staff guide is written); a decision-maker for a weekly 30-minute review.	Real-world validation before wide cutover; fast decisions.
6 · Business decisions	(a) Excel bulk-upload: keep or replace with the calendar tools? (b) Multi-currency: needed at launch or later? (c) B2B/DMC module timing. (d) SMS: wanted at launch, and via which provider? (e) Sign-off on the platform agreement & privacy text (legal).	Scope-freeze for the polish pack and back-office phases.
7 · Payment gateway later	Choice of gateway (PayHere / bank IPG / Stripe) + merchant account paperwork — only when we schedule the direct booking engine.	Booking-engine phase (growth layer).

THE TWO DATES THAT MATTER

Items 1 and 2 are the critical path. Every timeline in §15 assumes AxisRooms credentials and legacy DB access arrive in the first two weeks — each week they slip, go-live slips with them.

15. Time to complete the full product

Working assumptions: the current delivery setup (one senior engineer pairing with AI tooling — the same setup that built compartments A–K), client-side items from §14 arriving on time, and AxisRooms responding within normal certification timeframes. Estimates are given as ranges; phases marked || run in parallel with others.

#	Phase	Duration	Delivers
A	Go-live foundations	2–3 weeks	Staging + production hosting, deploy pipeline, domains/TLS, monitoring, tested backups, verified email domain. The demo system, on the internet.
B 	AxisRooms certification	2–4 weeks	Live channel-manager sync on a pilot property (mostly waiting on the external party — runs alongside A and C).
C	ETL & data migration	3–5 weeks	All legacy properties, rates, bookings and customers migrated; parity harness proving old = new to the cent; cutover runbook.
D	Staff back-office completion	4–6 weeks	Reference-data admin, per-OTA commissions, reconciliation + exports, campaign/notification engine.
E	Polish pack (P2 parity)	3–4 weeks	Seasons UI, My-channels screen, invoice PDF, si/ta translation, multi-currency display.
F	Booking engine + payments	4–6 weeks	Direct (commission-free) bookings on each hotel's own page, gateway integration, deposits.
G	Analytics + data warehouse	2–3 weeks	Occupancy / ADR / RevPAR dashboards; the aggregate layer the AI Revenue Manager needs.
H	AI Supplier Copilot v1	3–4 weeks	Chat assistant over the owner's own data (read-only, tenant-fenced tools). RM/Healer phases follow.

WHAT THAT ADDS UP TO

Production pilot live	≈ 6–8 weeks from kick-off (A + B + first ETL dry run; pilot hotels running in parallel with legacy)
Legacy fully replaced	≈ 3–4 months (all hotels cut over in waves, staff back-office complete — PMS, Extranet, Backend portal, Channel Manager, OTA all live on 2.0)
Full product incl. growth layer	≈ 6–7 months (adds booking engine + payments, analytics + warehouse, AI Copilot v1)

ILLUSTRATIVE CALENDAR (KICK-OFF AUGUST 2026)

Month	Focus	Milestone
Aug 2026	Hosting + deploy live; AxisRooms certification starts; ETL mapping & dry runs	System on real URLs; staging demo for staff
Sep 2026	Pilot hotels parallel-run (legacy + 2.0 side by side); staff back-office build	Pilot go-live ≈ 1 Oct
Oct 2026	Cutover waves; polish pack; back-office completion	Majority of hotels on 2.0
Nov 2026	Final wave; legacy read-only → retired; booking-engine build starts	Legacy switched off
Dec 26 – Jan 27	Booking engine + payments; analytics + warehouse; Copilot beta with pilot owners	Direct bookings taking payment
Feb 2027	AI Copilot v1 general release; RM roadmap begins	Full YB suite v1

Ranges, not promises: the two external dependencies (AxisRooms responsiveness, legacy data access) are the main variables — see §14.

16. Hosting, domain, database & product essentials

Philosophy: start simple, scale by stages. The application is stateless (all state lives in PostgreSQL/Redis/object storage), so we begin on modest managed infrastructure and grow it with revenue — no re-architecture required at any stage. Indicative list prices, July 2026; Rs figures at $\approx$ Rs 300/US\$.

Essential	Plan	Indicative cost
Domain & TLS	Keep <code>yohobed.com</code> ; subdomains <code>app.</code> (owners), <code>staff.</code> , <code>api.</code> . TLS certificates free (Let's Encrypt, auto-renewed). Cloudflare in front for DNS, CDN and basic WAF/rate-limiting.	$\sim$ \$15/yr if a new domain is wanted; Cloudflare \$0–20/mo
Application hosting	Stage 1: one production server (Docker: API + web + worker) in Singapore/Mumbai for Sri Lanka latency, plus a small staging server. DigitalOcean / AWS / Hetzner — the client owns the account.	\$50–100/mo production + $\sim$ \$20 staging
Database	Managed PostgreSQL 16 (the same engine the app runs on today) with automated daily backups, point-in-time recovery, and a weekly offsite dump to object storage. Start 2 vCPU / 4–8 GB; add a read replica at the growth stage.	\$50–100/mo; replica $\sim$ doubles it later
Cache / queue	Managed Redis (drives the channel-manager sync queue).	\$10–25/mo
Photo storage	Object storage (Cloudflare R2 or S3) behind the existing storage seam — one adapter swap, no app changes.	$\sim$ \$5/mo at launch
Email	Resend (already integrated — switching from demo mode is one environment variable) on a verified sending domain, e.g. <code>mail.yohobed.com</code> .	Free to 3k emails/mo; \$20/mo for 50k
SMS (optional)	Behind the same messaging seam; local aggregator (e.g. Dialog/Mobitel routes) or Twilio.	Pay-per-message ($\approx$ Rs 1–15 depending on route)
Monitoring & ops	Sentry error tracking, uptime checks, structured logs, alerting to email/WhatsApp. CI/CD via the GitHub Actions pipeline that already runs on every commit.	\$0–30/mo
Data warehouse	Stage 1: none needed — analytics read from a Postgres replica + nightly aggregate tables (this is also the substrate the AI Revenue Manager uses). Stage 2 (thousands of properties / heavy BI): ClickHouse or BigQuery with Metabase dashboards.	\$0 at launch $\rightarrow$ \$100–300/mo at scale
Security	Already in the product: database-enforced tenant isolation (RLS), RBAC, bcrypt, audit logs. Platform adds: WAF, least-privilege DB roles (already used), secrets management, tested restore drills.	Included / part of ops

WHY THIS IS SAFE TO START SMALL

Web and API are stateless — under load we add servers behind a load balancer, not redesign. Postgres carries this data shape comfortably for years (see §17); when reporting gets heavy we add a read replica before touching anything else. Every stage is an upgrade in place.

17. Scalability, the future of the YB suite & the budget reality

SCALABILITY — WITH REAL NUMBERS

- **The data is small by database standards.** 1,000 properties × 30 rooms × 365 priced days ≈ **11 million calendar rows a year** — routine for PostgreSQL with the indexes already in place. 100,000 bookings a year is trivial. The demo's whole dataset is a rounding error of what one Postgres server handles.
- **Multi-tenancy scales in one database.** Row-Level Security fences thousands of hotels in a single Postgres — the same model used by major SaaS platforms — with sharding by tenant available as a far-future option, not a prerequisite.
- **Spikes are absorbed by the queue.** A price change across 60 days × 500 properties doesn't hammer the OTAs — it lands in the outbox and the worker drains it with retries. Add worker processes to drain faster; the API never blocks on the channel manager.
- **Stateless web/API tier** = horizontal scale by adding instances behind a load balancer; sessions are JWTs, uploads are in object storage, nothing lives on the server.
- **Proof culture:** the concurrency-sensitive paths (overbooking, booking references) are guarded by automated race-condition tests, so scale doesn't reintroduce the legacy bugs.

THE FUTURE OF THE YB SUITE

The market context: channel managers and PMSes (SiteMinder, STAAH, Cloudbeds and peers) typically charge hotels on the order of **US\$50–150+ per property per month**, none of them are AI-native, and none offer Sri Lankan-market commission-based economics with local support. YoHoBed 2.0's roadmap is built to win on exactly those axes:

Wave	What it adds
Now — Operations	The PMS you demoed: run the hotel, distribute to OTAs, settle transparently.
Next — Revenue	Direct booking engine (commission-free direct sales) + payments; analytics (occupancy/ADR/RevPAR).
Then — Intelligence	AI Supplier Copilot: every owner gets an analyst in their pocket — "How did August compare to last year? Which weekends should I reprice?" — answered from their own live data, read-only and tenant-fenced. Then the AI Revenue Manager (pricing suggestions on the analytics substrate) and the Distribution Healer (spots and fixes channel problems before the owner notices).
Beyond — Network	B2B/DMC marketplace (YoHoBed's bedbank roots), two-way guest messaging, mobile PWA, agentic distribution — inventory bookable by AI travel agents, a channel none of the incumbents serve yet.

AI RUNNING COST — A WORKED EXAMPLE

A typical Copilot question consumes roughly 10,000 input + 1,000 output tokens. On Claude Haiku 4.5 (US\$1 in / \$5 out per million tokens) that is ≈ **\$0.015 per question**; on Claude Sonnet 5 (\$3/\$15) for complex analysis ≈ \$0.045. An owner asking 100 questions a month costs ≈ **\$1.50–4.50/property/month** — comfortably packaged inside a premium tier, and a feature competitors don't have at any price.

BUDGET REALITY — INFRASTRUCTURE RUN-RATE BY STAGE

Stage	Scale	Monthly infra + services	≈ LKR / month	Per property
Pilot / launch	up to ~100 properties	\$150–250	Rs 45k–75k	\$1.50–2.50
Growth	100–500 properties	\$400–700 (replica, bigger DB, LB)	Rs 120k–210k	~\$1.40
Scale	500–2,000 properties	\$900–1,500 (multi-instance, warehouse)	Rs 270k–450k	~\$0.75
Big	5,000+ properties	\$2,500–4,000	Rs 0.75m–1.2m	<\$0.80

The punchline: at growth scale the platform costs **≈ US\$1.40 per property per month** to run — while the incumbents charge hotels 50–100× that. Infrastructure will never be the constraint on this business; it scales sub-linearly with revenue.

One-time costs	Domain ~\$15/yr; TLS free; AxisRooms certification is effort, not fees, on our side. No licence fees anywhere in the stack — every component (PostgreSQL, Redis, NestJS, Next.js) is open-source.
Year-1 infra example	Pilot-stage average ≈ \$200/mo → ≈ \$2,400 (~Rs 720k) for the first year of hosting, database, email, monitoring and backups — excluding development effort (phased per \$15, commercial terms per our engagement) and optional SMS volume.
Caveats	Indicative public list prices as of July 2026; Rs figures at ≈ Rs 300/US\$; AI costs billed per actual usage (worked example above); prices re-confirmed at account setup inside the client's own cloud account — you see every bill.